

MULTI-VENDOR LOCAL MARKETPLACE

A Shop-Based E-Commerce Platform for Local Wholesalers & Brand Retailers

Project Report — Content for Presentation

1. Introduction

Most e-commerce platforms sell products in a single flat catalogue, which does not fit how local wholesale and brand-based retail actually works. In many local markets, a shop owner sells a specific set of branded products under their own shop identity — a customer wants that grocer's brand, not a random seller's version of the same item.

This project is a multi-vendor marketplace where every approved seller runs an independent virtual shop. Products uploaded by a seller are visible only within that seller's own shop — they never mix with another seller's catalogue. Buyers browse a directory of shops, enter a specific shop, and shop only within that shop's product range, similar to visiting a physical store.

The platform is aimed at local wholesalers, brand owners, and neighbourhood shop owners (e.g. a grocery store that stocks its own branded products) who want an online storefront that customers can find and order from directly, without competing in a mixed marketplace listing.

2. Objectives

- Let sellers register, get admin-approved, and run their own independent shop.
- Keep each shop's product catalogue isolated and visible only within that shop.
- Let buyers discover shops, browse products by shop, add to cart, and place orders.
- Keep the stack minimal and practical: plain React (JavaScript), Node/Express, MongoDB only.
- Support image-heavy product catalogues without slowing the platform down.

3. Technology Stack

Layer	Technology	Why
Frontend	React.js (JavaScript)	Fast dev server, no TypeScript overhead, component-based UI
Styling	Tailwind CSS	Utility-first, fast to build with, no separate CSS files to manage
State management	React Context API + useReducer	Enough for cart/auth state; avoids Redux boilerplate
Backend	Node.js + Express.js	JS everywhere (MERN-style), simple REST API layer
Database	MongoDB	NoSQL, schema-flexible — fits shops/products/orders well
Authentication	JWT + bcrypt	Stateless auth, role-based access (buyer / seller / admin)
Image hosting/CDN	Cloudinary	Free tier, auto image optimization + CDN delivery
Payments	Razorpay Payment Links	India-first, quick integration, no PCI burden
Deployment	Render / Vercel	Free-tier friendly, auto-deploy from Git
Load handling for uploads	Nginx reverse proxy + Cloudinary CDN	Offloads image serving from the app server

Note: only MongoDB is used for data storage — no SQL database anywhere in the stack.

4. User Roles

Role	What they do
Buyer	Browse the shop directory, open a shop, view its products, add to cart, place orders, track order status.
Seller	Register → wait for admin approval → set up shop profile → upload/manage products → view & fulfil orders for their own shop.
Admin	Approve or reject new seller/shop applications, manage categories, moderate listings, view platform-wide orders.

5. Core Features (used in this project only)

5.1 Buyer-side

- Browse the list of all approved shops
- Open a specific shop and view only that shop's products
- Search and filter products within a shop
- Add products to cart (cart is per-shop or multi-shop split at checkout)
- Place an order and track its status (placed → confirmed → shipped → delivered)
- Buyer profile with order history

5.2 Seller-side

- Seller registration with shop details (shop name, brand, address)
- Approval workflow — shop goes live only after admin approval
- Seller dashboard: add / edit / delete products with images
- Stock and price management per product
- View and update incoming orders for their shop
- Shop profile page shown publicly to buyers

5.3 Admin-side

- Approve or reject seller/shop registration requests
- Manage product categories
- View all shops, all orders, and basic platform stats
- Suspend/remove a shop if needed

5.4 Cross-cutting

- JWT-based authentication with role-based route protection
- Shop-scoped product isolation (a core rule enforced at the database + API level)
- Image upload via Multer + Cloudinary for product photos and shop logos
- Cart and checkout flow with order creation

6. Database Design (MongoDB)

Each collection below is a Mongoose model. Relationships are done via ObjectId references, kept minimal since MongoDB is schema-flexible.

6.1 users

Field	Type	Notes
id	ObjectId	Primary key
name	String	
email	String	Unique, used for login
password	String	bcrypt hashed
role	String	'buyer' 'seller' 'admin'
phone	String	
address	String	Default delivery address for buyers
createdAt	Date	

6.2 shops

Field	Type	Notes
id	ObjectId	Primary key
sellerId	ObjectId (ref: users)	Owner of the shop
shopName	String	
shopLogo	String	Cloudinary URL
description	String	
address	String	Local area / city for discovery
isApproved	Boolean	Set by admin before the shop goes live
createdAt	Date	

6.3 products

Field	Type	Notes
id	ObjectId	Primary key
shopId	ObjectId (ref: shops)	Ties product to exactly one shop — this is what enforces isolation
name	String	
brand	String	
description	String	
price	Number	
stock	Number	
images	[String]	Cloudinary URLs
category	String	
createdAt	Date	

6.4 carts

Field	Type	Notes
id	ObjectId	Primary key
buyerId	ObjectId (ref: users)	
items	[{ productId, shopId, quantity }]	Embedded array
updatedAt	Date	

6.5 orders

Field	Type	Notes
id	ObjectId	Primary key
buyerId	ObjectId (ref: users)	

Field	Type	Notes
shopId	ObjectId (ref: shops)	One order belongs to one shop
products	[{ productId, name, price, quantity }]	Snapshot at order time
totalAmount	Number	
status	String	'placed' 'confirmed' 'shipped' 'delivered' 'cancelled'
deliveryAddress	String	
createdAt	Date	

6.6 categories

Field	Type	Notes
id	ObjectId	Primary key
name	String	Managed by admin

7. Project Folder Structure

A matching empty folder scaffold is provided alongside this report as a ZIP — extract it and start filling in files.

```

multi-vendor-marketplace/
├── client/                                React frontend (JavaScript)
│   ├── public/
│   └── src/
│       ├── assets/
│       ├── components/                Reusable UI: Navbar, ProductCard, ShopCard, CartItem...
│       ├── pages/
│       │   ├── buyer/                ShopList, ShopPage, Cart, Checkout, Orders
│       │   ├── seller/                Dashboard, AddProduct, MyOrders, ShopSettings
│       │   └── admin/                  ApproveShops, ManageCategories, AllOrders
│       ├── context/                   AuthContext, CartContext
│       ├── services/                  api.js (axios instance), auth.js, product.js, order.js
│       ├── utils/
│       ├── App.jsx
│       └── main.jsx
├── server/                              Express backend
│   ├── config/                        db.js, cloudinary.js
│   ├── models/                        User.js, Shop.js, Product.js, Cart.js, Order.js, Category.js
│   ├── controllers/                   authController.js, shopController.js, productController.js,
│   │                                   cartController.js, orderController.js, adminController.js
│   ├── routes/                        authRoutes.js, shopRoutes.js, productRoutes.js,
│   │                                   cartRoutes.js, orderRoutes.js, adminRoutes.js
│   ├── middleware/                    authMiddleware.js, roleMiddleware.js, uploadMiddleware.js
│   ├── utils/                          generateToken.js
│   ├── server.js
│   └── .env
└── docs/                              This report, ER diagram, screenshots for the presentation

```

8. Image Management & Upload Handling

- Product/shop images are uploaded from React → sent as multipart/form-data → Express receives via Multer.
- Multer forwards the file buffer to Cloudinary; Cloudinary returns a CDN URL, which is what gets stored in MongoDB (never the raw file).
- This keeps the database light — only URLs are stored, not binary image data.
- Cloudinary's CDN serves the images directly to buyers' browsers, so the Node server itself never becomes a bottleneck for image traffic — this is the 'load balancing' for uploads: the heavy lifting is offloaded away from your own server.
- Optional: an Nginx reverse proxy in front of the Node server if you self-host, to handle concurrent connections more gracefully.

9. Suggested Presentation Format

A practical slide order to build from this report:

- Slide 1 — Title (project name, your name, college)
- Slide 2 — Problem Statement (Section 1)
- Slide 3 — Objectives (Section 2)
- Slide 4 — Tech Stack (Section 3 table)
- Slide 5 — User Roles (Section 4 table)
- Slide 6-9 — Features, one slide per role (Section 5)
- Slide 10 — Database Design / ER-style diagram (Section 6)
- Slide 11 — Folder/Architecture Structure (Section 7)
- Slide 12 — Image Upload Flow diagram (Section 8)
- Slide 13 — Future Scope (ratings/reviews, delivery tracking, analytics dashboard)
- Slide 14 — Thank You / Q&A

10. Out of Scope (deliberately not included)

- No SQL database of any kind — MongoDB only, as specified.
- No TypeScript, no Next.js/Remix — plain React + Vite only.
- No reviews/ratings, wishlist, or coupon system in v1 — these were not requested and can be added later.